

Busca em árvore Monte Carlo II: AlphaZero

PUCT • auto-jogo • rede política-valor • AlphaTensor e AlphaDev • raciocínio metanível

Prof. Marcelo Keese Albertini

Adaptado de CS234: Reinforcement Learning, Emma Brunskill, Stanford University — Lecture 14

Intuição

Como usar este handout. Ele não resume os slides: complementa-os. O que no projetor aparece como resultado, aqui aparece com a passagem inteira — a derivação da forma fechada por trás do PUCT, a conta da densidade de informação no auto-jogo, o traço numérico completo da busca. As perguntas em Para discutir em aula são as que o professor conduz oralmente; os exercícios ao final incluem itens que não estão no CS234.

1 Decidir agora, num mundo grande demais

1.1 Duas maneiras de responder “o que eu faço?”

Até esta altura do curso, quase todo algoritmo produziu o mesmo tipo de objeto: uma política $\pi : S \rightarrow \mathcal{A}$ definida sobre *todo* o espaço de estados. Iteração de valor, Q -learning, DQN, REINFORCE, PPO — todos entregam uma função que responde a qualquer estado que apareça. Chame isso de resposta *amortizada*: paga-se caro uma vez, na fase de treino, e cada consulta posterior é barata.

Existe outra maneira. Dado o estado atual s_t , gastar computação *agora* para escolher bem *uma* ação, e só ela. Depois, no próximo estado, repetir do zero. Chame isso de resposta *sob demanda*: nada é pago antecipadamente, mas cada decisão custa.

A escolha entre as duas não é estética. Quando $|S|$ é astronômico — e 10^{170} posições legais no Go é astronômico em qualquer escala razoável — a primeira estratégia é literalmente impossível: não há memória no universo para tabular π , nem dados para treinar uma aproximação uniformemente boa. Sobra a segunda. E aí se descobre algo mais interessante: as duas não competem. AlphaZero usa a busca local para *fabricar dados* com que treina uma política global, e usa a política global para *acelerar* a busca local. O sistema inteiro é esse laço.

1.2 Busca Monte Carlo simples, e por que ela não basta

O algoritmo mais direto: dado um simulador $\mathcal{M}$ e uma política de simulação π , para cada ação a simule K episódios a partir de s_t e estime

$$\hat{Q}(s_t, a) = \frac{1}{K} \sum_{k=1}^K G_t^k \xrightarrow{K \rightarrow \infty} Q^\pi(s_t, a), \quad a_t = \arg \max_a \hat{Q}(s_t, a).$$

Vale a pena ser explícito sobre o que isso é e o que não é. O estimador converge para Q^π , não para Q^* . Escolher $\arg \max_a Q^\pi(s_t, a)$ é exatamente um passo de *melhoria de política* sobre π . Pelo teorema da melhoria de política, a política resultante é pelo menos tão boa quanto π — mas apenas um passo à frente dela.

Atenção

Se π for a política aleatória uniforme, a decisão será “a melhor jogada supondo que, a partir do lance seguinte, ambos os jogadores joguem ao acaso”. Em Go isso é quase inútil: a maioria das posições tem valor semelhante sob jogo aleatório. É por essa razão que a MCTS existe — para fazer *mais* de um passo de melhoria.

1.3 A árvore expectimax e a explosão combinatória

Com um modelo, dá para calcular $Q^*(s_t, a)$ exatamente, expandindo a árvore completa a partir de s_t e retropropagando

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a').$$

A observação útil é que *não é preciso resolver o MDP inteiro*: basta o sub-MDP que começa em s_t e vai até o horizonte. A observação fatal é que esse sub-MDP tem tamanho $(|S||A|)^H$.

	ramificação	duração	folhas da árvore completa
Xadrez	≈ 35	≈ 80	$\approx 10^{123,5}$
Go	≈ 250	≈ 150	$\approx 10^{359,7}$
átomos no universo observável			$\approx 10^{80}$

1.4 MCTS: crescer a árvore só onde importa

A resposta da busca em árvore Monte Carlo é não construir a árvore toda. Constrói-se uma árvore *assimétrica*, aprofundada apenas nas regiões que as simulações anteriores indicaram valer a pena. Cada simulação tem quatro fases:

- Seleção.** A partir da raiz, descer pela árvore existente escolhendo, em cada nó, a ação que maximiza um critério que combina valor estimado e incerteza.
- Expansão.** Ao chegar a um nó que ainda não está na árvore, acrescentá-lo.
- Avaliação.** Obter um valor v para esse nó novo — por rollout (MCTS clássica) ou por uma rede (AlphaZero).
- Retropropagação.** Subir pelo caminho percorrido atualizando, em cada aresta (s, a) : $N += 1$, $W += v$, $Q = W/N$.

Intuição

Todo o projeto de um algoritmo de MCTS está em **duas decisões**: o critério da fase 1 e a origem do v da fase 3. Tudo o mais é contabilidade. Vale a pena carregar esse par como lente ao ler qualquer variante da literatura.

1.5 UCT, e um detalhe a guardar

O UCT (Kocsis e Szepesvári, 2006) resolve a fase 1 tratando cada nó como um bandit de múltiplos braços:

$$a_{ik} = \arg \max_a \left[\frac{1}{N(i, a)} \sum_{k=1}^{N(i, a)} G_k(i, a) + c \sqrt{\frac{\log N(i)}{N(i, a)}} \right].$$

Duas consequências merecem destaque, porque voltarão na Seção 6.

Primeiro, a **política de simulação muda a cada episódio**: a árvore não é percorrida por uma política fixa, mas por uma que se adapta ao histórico. Segundo, e como corolário direto, a **média $\bar{G}$ retropropagada é uma média sobre uma política não estacionária**. Ela não estima Q^* nem Q^π para nenhum π fixo; estima a média histórica de uma sequência de políticas cada vez melhores. Converge para o valor ótimo, sim, mas por baixo e devagar — e o atraso de cada nível se compõe ao subir pela árvore.

2 AlphaZero: a busca guiada por uma rede

2.1 Uma rede, duas cabeças

Uma única rede residual f_θ recebe a posição s e devolve o par

$$f_\theta(s) = (p_\theta(s), v_\theta(s)), \quad p_\theta(s) \in \Delta(\mathcal{A}), \quad v_\theta(s) \in [-1, 1].$$

A cabeça de política é usada como *prior* na fase de seleção; a cabeça de valor substitui a rollout na fase de avaliação. O corpo é compartilhado porque as duas tarefas exigem as mesmas representações: reconhecer grupos vivos e mortos, influência, forma.

Uma passagem direta contra cem rollouts

No AlphaGo de 2016, uma única passagem de v_θ atingiu erro quadrático médio comparável à média de 100 rollouts com a política de RL — a um custo ordens de grandeza menor. Esse número explica a trajetória posterior: a otimização bayesiana dos hiperparâmetros do AlphaGo deslocou progressivamente o peso da mistura em direção à rede de valor a cada ciclo de projeto, até a rollout ser removida por completo no AlphaGo Zero (Chen et al., 2018).

2.2 PUCT: anatomia da fórmula

Em cada nó, mantêm-se por ação: $N(s, a)$, $W(s, a)$, $Q(s, a) = W/N$ e o prior $P(s, a) = p_\theta(s)_a$. A ação a simular é

Regra de seleção do AlphaZero

$$a_t = \arg \max_a [Q(s, a) + U(s, a)], \quad U(s, a) = c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}.$$

Leia termo a termo. O numerador $\sqrt{N(s)}$ faz o bônus *crescer* com a quantidade total de trabalho já feito no nó — quanto mais o nó foi visitado, mais se justifica reexaminar alternativas. O denominador $1 + N(s, a)$ faz o bônus *cair* com o trabalho já feito naquela ação específica. O fator $P(s, a)$ pondera tudo pela opinião da rede. E o +1 no denominador garante que uma ação nunca visitada tenha bônus *finito*, igual a $c_{\text{puct}} P(s, a) \sqrt{N(s)}$ — diferente do UCT, em que ações não visitadas têm bônus infinito e são todas tentadas obrigatoriamente.

	UCT	PUCT
Bônus	$c\sqrt{\log N(s)/N(s, a)}$	$c_{\text{puct}} P(s, a) \sqrt{N(s)} / (1 + N(s, a))$
Decaimento em $N(s, a)$	$N(s, a)^{-1/2}$	$N(s, a)^{-1}$
Crescimento em $N(s)$	$\sqrt{\log N(s)}$	$\sqrt{N(s)}$
Ação nunca visitada	bônus infinito	bônus finito, $\propto P(s, a)$
Justificativa	desigualdade de Hoeffding	heurística
Avaliação da folha	rollout até o fim	v_θ

Atenção

O PUCT não é uma cota superior de confiança. Não há desigualdade de concentração por trás dele, e a consequência é operacional: *ações com prior baixo podem nunca ser visitadas*. Em Go, com ≈ 250 jogadas legais e 1 600 simulações, é isso que torna a busca viável — e é isso que exige o ruído de Dirichlet da Seção 2.6.

2.3 O PUCT em câmera lenta

Considere uma raiz com três jogadas em que a rede erra: dá prior alto para a_1 , que é a pior das duas jogadas razoáveis. Priors $P = (0,60; 0,30; 0,10)$; valores das folhas $v_\theta = (0,10; 0,55; -0,40)$; $c_{\text{puct}} = 1,5$. Na primeira simulação, com $N(s) = 0$, adota-se a convenção $\sqrt{N(s)} \rightarrow 1$.

k	$Q(s, a)$			$U(s, a)$			$Q + U$			escolhe
	a_1	a_2	a_3	a_1	a_2	a_3	a_1	a_2	a_3	
1	0,00	0,00	0,00	0,900	0,450	0,150	0,900	0,450	0,150	a_1
2	0,10	0,00	0,00	0,450	0,450	0,150	0,550	0,450	0,150	a_1
3	0,10	0,00	0,00	0,424	0,636	0,212	0,524	0,636	0,212	a_2
4	0,10	0,55	0,00	0,520	0,390	0,260	0,620	0,940	0,260	a_2
5	0,10	0,55	0,00	0,600	0,300	0,300	0,700	0,850	0,300	a_2
6	0,10	0,55	0,00	0,671	0,252	0,335	0,771	0,802	0,335	a_2
7	0,10	0,55	0,00	0,735	0,220	0,367	0,835	0,770	0,367	a_1
8	0,10	0,55	0,00	0,595	0,238	0,397	0,695	0,788	0,397	a_2
9	0,10	0,55	0,00	0,636	0,212	0,424	0,736	0,762	0,424	a_2
10	0,10	0,55	0,00	0,675	0,193	0,450	0,775	0,743	0,450	a_1

Quatro leituras, na ordem em que aparecem.

Simulações 1–2: a rede manda sozinha. Nada foi avaliado, $Q \equiv 0$, e o ranking é o do prior. Esta é a fase em que a qualidade de p_θ é tudo.

Simulação 3: a evidência entra. Assim que a_1 devolve 0,10 — abaixo do que o prior insinuava — o somatório muda de líder. Note que bastou *uma* avaliação para virar o jogo; é o termo U decaindo como $N(s, a)^{-1}$ que permite essa rapidez.

Simulação 7: a busca volta a conferir a_1 . O denominador $1 + N(s, a_2)$ cresceu o bastante para que a_1 recupere a liderança momentaneamente. Isto não é desperdício: é o mecanismo que impede a busca de se fechar em torno de um erro precoce.

A ação a_3 fica fora até $k = 23$. Prior baixo mais orçamento pequeno é igual a jogada nunca examinada. Com 16 simulações, $N = (5, 11, 0)$.

2.4 Quando o prior está muito errado

O traço acima é benigno: o prior erra, mas não muito. O caso preocupante é o prior confiante e errado. Mantendo v_θ e trocando os priors para $P = (0,95; 0,04; 0,01)$:

simulações	$N(a_1)$	$N(a_2)$	$N(a_3)$	recomendação da raiz
50	33	17	0	a_1 — errado
200	44	156	0	a_2
800	89	711	0	a_2

A busca corrige a rede, mas o preço é orçamento — e por um bom tempo ela *confirma* o erro em vez de corrigi-lo. Esta é a formulação quantitativa do princípio geral: rede melhor significa

menos simulações para a mesma decisão. É também a razão pela qual a curva de “força em função do número de simulações” tem um joelho: até certo ponto, mais busca compra correção do prior; depois disso, compra pouco.

2.5 c_{puct} : quanto duvidar da própria rede

Mesmo exemplo, orçamento fixo de 400 simulações, variando só c_{puct} :

c_{puct}	$N(a_1)$	$N(a_2)$	$N(a_3)$	interpretação
0,5	13	386	1	confia na rede; converge rápido
1,0	25	373	2	faixa típica na prática
1,5	37	360	3	
3,0	71	324	5	
5,0	107	284	9	duvida da rede; gasta reexaminando o descartado

Intuição

Não é um parâmetro cosmético. No AlphaGo, ajustar por otimização bayesiana os hiperparâmetros da MCTS — c_{puct} entre eles — rendeu **94 e 103 pontos de Elo** em dois ciclos de projeto antes da partida contra Lee Sedol. Cem pontos de Elo correspondem a cerca de 64% de vitórias contra a versão anterior. Ao ler comparações entre algoritmos de MCTS na literatura, pergunte sempre quanto de ajuste houve de cada lado.

2.6 Ruído de Dirichlet

Se o prior é o que abre a porta, e a porta de a_3 nunca abre, o sistema não tem como descobrir que a rede está errada sobre a_3 . Em auto-jogo — e apenas em auto-jogo — o AlphaZero perturba o prior na raiz:

$$P(s_0, a) = (1 - \varepsilon) p_{\theta}(s_0)_a + \varepsilon \eta_a, \quad \eta \sim \text{Dir}(\alpha), \quad \varepsilon = 0,25.$$

O parâmetro α é calibrado inversamente ao número típico de jogadas legais: 0,30 para xadrez (≈ 35 jogadas), 0,15 para shogi (≈ 90) e 0,03 para Go (≈ 250).

Vale entender por que α pequeno. Uma Dirichlet com $\alpha \ll 1$ produz vetores *esparcos*: quase toda a massa cai em uma ou duas coordenadas sorteadas. O efeito é que, em cada partida de auto-jogo, um punhado de jogadas recebe um empurrão *grande*, em vez de todas receberem um empurrão minúsculo. Isso gera exploração profunda — a jogada sorteadas é realmente investigada — em vez de uma diluição uniforme que não muda nada.

Intuição

O ruído fica só na raiz, e isso é essencial. A busca precisa *poder* tentar a jogada estranha, mas continua livre para descartá-la assim que Q mostrar que é ruim. É exploração que a evidência pode vetar — diferente de ϵ -guloso, que força a execução da ação aleatória. Em avaliação e partidas oficiais, o ruído é removido: ele é um mecanismo de *coleta de dados*, não de jogo.

2.7 A política da raiz e a temperatura

Terminadas as K simulações, a política exportada é

$$\pi(a | s_0) = \frac{N(s_0, a)^{1/\tau}}{\sum_b N(s_0, b)^{1/\tau}}.$$

Com $N = (5, 11, 0)$ do traço da Seção 2.3:

τ	$\pi(a_1)$	$\pi(a_2)$	$\pi(a_3)$	$H[\pi]$ (bits)
1,00	0,3125	0,6875	0	0,896
0,50	0,1712	0,8288	0	0,661
0,25	0,0409	0,9591	0	0,247
0,10	0,0004	0,9996	0	0,005

No auto-jogo usa-se $\tau = 1$ nas 30 primeiras jogadas — para gerar diversidade de posições no conjunto de treino — e $\tau \rightarrow 0$ depois, para jogar a partida com seriedade.

Por que exportar contagens de visita em vez de $\arg \max_a Q(s_0, a)$, que parece a estimativa mais direta? Porque N é um resumo mais robusto. Uma ação visitada três vezes pode ter Q alto por sorte; Q sozinho não carrega informação sobre quanta evidência o sustenta. As contagens já incorporam a confiança: só recebe muitas visitas o que sobreviveu à comparação repetida com as alternativas ao longo de toda a busca.

2.8 O que o PUCT está de fato otimizando

As contagens de visita não são um subproduto arbitrário da regra de seleção. Grill et al. (ICML 2020) mostraram que a distribuição de visitas normalizada resolve aproximadamente um problema de otimização de política regularizada por divergência de Kullback–Leibler.

Demonstração

Problema. Com q os valores estimados e p_θ o prior, considere

$$\bar{\pi} = \arg \max_{y \in \Delta(\mathcal{A})} \left[q^\top y - \lambda \text{KL}(p_\theta \| y) \right], \quad \lambda > 0.$$

Derivação. Escrevendo $\text{KL}(p_\theta \| y) = \sum_a p_a \log p_a - \sum_a p_a \log y_a$, o primeiro somatório não depende de y e pode ser descartado. O lagrangiano com o vínculo $\sum_a y_a = 1$ é

$$\mathcal{L}(y, \alpha) = \sum_a q_a y_a + \lambda \sum_a p_a \log y_a - \alpha \left(\sum_a y_a - 1 \right).$$

Derivando em y_a e igualando a zero:

$$q_a + \frac{\lambda p_a}{y_a} - \alpha = 0 \quad \Rightarrow \quad y_a = \frac{\lambda p_a}{\alpha - q_a}$$

com α determinado pela normalização $\sum_a \lambda p_a / (\alpha - q_a) = 1$. Como o logaritmo força $y_a > 0$, é necessário $\alpha > \max_a q_a$; o lado esquerdo é estritamente decrescente em α nesse intervalo e vai de $+\infty$ a 0, logo a raiz existe e é única.

Casos-limite. Quando $\lambda \rightarrow \infty$, a normalização exige $\alpha \rightarrow \infty$ e $y_a \rightarrow p_a$: recupera-se o prior. Quando $\lambda \rightarrow 0^+$, tem-se $\alpha \rightarrow (\max_a q_a)^+$, e a coordenada que atinge o máximo domina: $y \rightarrow$ indicadora do $\arg \max$ de q .

A conexão com o PUCT é a identificação $\lambda_N = c_{\text{puct}} \sqrt{N} / (N + |\mathcal{A}|)$ e $\bar{\pi}_a \approx N(s, a) / (N + |\mathcal{A}|)$. A tabela abaixo executa o PUCT do exemplo da Seção 2.3 e compara com a forma fechada:

N	visitas empíricas			forma fechada			λ_N
	a_1	a_2	a_3	a_1	a_2	a_3	
64	0,1940	0,7463	0,0149	0,2069	0,7755	0,0176	0,1791
256	0,1120	0,8687	0,0077	0,1154	0,8752	0,0094	0,0927
1600	0,0480	0,9464	0,0037	0,0486	0,9475	0,0039	0,0374
6400	0,0245	0,9731	0,0019	0,0247	0,9734	0,0020	0,0187

O acordo é bom já em $N = 256$ e excelente daí em diante. A leitura conceitual: como $\lambda_N \sim N^{-1/2} \rightarrow 0$, a regularização enfraquece com o orçamento. Pouco orçamento significa “fique perto do que a rede acha”; muito orçamento significa “vá para o $\arg \max$ dos valores”. É a mesma estrutura de proximidade regularizada do TRPO e do PPO — só que aqui ela emerge de um planejador, sem ter sido escrita em lugar nenhum.

Para discutir em aula

O PUCT foi projetado como heurística e só treze anos depois foi reinterpretado como otimização regularizada. Isso é comum em aprendizagem de máquina. Que outros exemplos do curso seguem o mesmo padrão — heurística primeiro, justificativa depois? A justificativa *a posteriori* tem valor prático, ou é apenas arrumação estética?

3 Auto-jogo: o laço que fabrica os próprios dados

3.1 A estrutura do laço

O sistema completo é um ciclo de quatro etapas, sem nenhum humano dentro: a busca guiada por f_θ produz π ; a partida é jogada amostrando de π ; cada jogada vira um exemplo (s_t, π_t, z) ; a rede é treinada nesses exemplos; e a rede nova volta a guiar a busca.

O único insumo externo são as *regras do jogo*, usadas pelo simulador para gerar sucessores e detectar o fim da partida.

Partidas por iteração	25 000
Simulações por jogada	1600 ($\approx 0,4$ s por busca)
Temperatura	$\tau = 1$ nas 30 primeiras jogadas, depois $\tau \rightarrow 0$
Ruído na raiz	$\varepsilon = 0,25$, $\eta \sim \text{Dir}(0,03)$ no Go
Desistência	automática, mantendo falsos positivos abaixo de 5%
Rodada de 3 dias	4,9 milhões de partidas, rede de 20 blocos
Rodada de 40 dias	29 milhões de partidas, rede de 40 blocos, Elo 5 185

3.2 A perda, e a assimetria entre os dois alvos

Função de perda

$$\ell(\theta) = \underbrace{(z - v_\theta(s))^2}_{\text{regressão do valor}} - \underbrace{\pi^\top \log p_\theta(s)}_{\text{entropia cruzada}} + \underbrace{c \|\theta\|^2}_{\text{regularização}}$$

Os dois alvos têm naturezas muito diferentes, e confundi-los é fonte comum de mal-entendidos.

O **alvo de política** π_t é *denso e rico*. Uma partida de 200 jogadas produz 200 alvos, e cada um é uma distribuição inteira sobre as jogadas legais — não um rótulo duro. É o resultado do passo de melhoria sendo destilado na rede.

O **alvo de valor** z é *escasso e correlacionado*: um único bit por partida, replicado em todas as posições dela. Duas posições da mesma partida compartilham exatamente o mesmo rótulo, ainda que uma seja do lance 10 e a outra do lance 190.

Atenção

Diz-se com frequência que “Go é um problema de recompensa esparsa”. É meia verdade. A *recompensa* é terminal, mas o *alvo de treino da política* é denso, porque a busca fabrica supervisão em cada posição. É um dos motivos pelos quais o auto-jogo com MCTS aprende, enquanto gradiente de política puro com recompensa terminal patinaria: a MCTS converte um sinal terminal em supervisão por jogada.

3.3 Currículo automático e densidade de informação

O adversário do AlphaZero é uma cópia de si mesmo. A consequência é que a taxa de vitória fica em torno de 50% durante todo o treino, e isso é mais importante do que parece.

Trate o resultado z como uma variável de Bernoulli com parâmetro p igual à probabilidade de vitória. A informação que uma partida carrega sobre os parâmetros é máxima quando a entropia de z é máxima:

p (prob. de vitória)	$H(z)$ em bits	partidas por bit de informação
0,50	1,000	1,0
0,75	0,811	1,2
0,90	0,469	2,1
0,99	0,081	12,4

Contra um adversário fixo e muito mais forte, todas as partidas terminariam em derrota: $z \equiv -1$, entropia zero, gradiente do termo de valor praticamente nulo. Contra um muito mais fraco, o mesmo com o sinal trocado. Um adversário sempre à altura é a definição operacional de um *currículo*: a dificuldade sobe exatamente na velocidade da competência.

Intuição

Esta é uma resposta parcial — e desconfortável — à pergunta “por que RL funciona tão bem em jogos e sofre tanto no mundo real”. Jogos de dois jogadores vêm com um gerador de currículo embutido, de graça. Robótica, medicina e diálogo não vêm. Onde não há auto-jogo, o currículo tem de ser construído à mão, e isso é engenharia difícil.

Para discutir em aula

Existem domínios não competitivos em que se pode fabricar um análogo do auto-jogo? Pense em geração de problemas cuja dificuldade acompanha a competência atual do agente — provas matemáticas, casos clínicos sintéticos, cenários de tráfego. O que faz o papel do “adversário à altura” nesses casos, e o que se perde em relação ao jogo de dois jogadores?

4 O que a evidência mostra

O artigo do AlphaGo Zero é incomum por trazer ablações limpas. Três perguntas foram isoladas: quanto do avanço vem da arquitetura, quanto vem da busca, e quanto se perde ao dispensar dados humanos.

4.1 Arquitetura

Quatro redes foram treinadas no mesmo conjunto fixo de partidas de auto-jogo, combinando política e valor *separadas* (sep) ou *conjuntas* (dual), e arquitetura *convolucional* (conv) ou *residual* (res). Todas foram avaliadas com a mesma busca, 5 s por jogada.

Configuração	Ganho em Elo	Corresponde a
sep-conv	base	arquitetura do AlphaGo Lee
dual-conv	+600	
sep-res	+600	arquitetura do AlphaGo Zero
dual-res	+1 200	

Cada uma das duas mudanças vale, sozinha, mais de 600 Elo; juntas, mais de 1200 — a distância entre um forte amador e um profissional. É um resultado desconfortável e honesto: parte substancial do avanço rotineiramente atribuído “ao algoritmo” era engenharia de rede. Curiosamente, a cabeça conjunta *piora* ligeiramente a previsão de jogadas humanas; em compensação reduz o erro de valor e fortalece o jogo. Prever bem o humano e jogar bem não são o mesmo objetivo.

4.2 Busca

A mesma rede treinada, jogando sem nenhum lookahead — apenas $\arg \max_a p_\theta(s)_a$ — contra o sistema completo:

Jogador	Elo	
Rede crua do AlphaGo Zero (sem busca)	3 055	
AlphaGo Fan	3 144	venceu Fan Hui
AlphaGo Lee	3 739	venceu Lee Sedol
AlphaGo Master	4 858	60–0 contra profissionais on-line
AlphaGo Zero	5 185	100–0 contra o AlphaGo Lee

A diferença entre a rede crua e o sistema completo é de 2 130 pontos. Na escala Elo, a probabilidade de vitória do mais forte é $p = 1/(1 + 10^{-\Delta/400})$, o que dá:

Δ Elo	p (vitória)	derrota a cada
200	0,7597	4 partidas
600	0,9693	33 partidas
1 200	0,9990	1 001 partidas
2 130	0,999995	211 350 partidas

Busca é computação em tempo de decisão

A mesma rede, com e sem MCTS, difere em mais de 2 000 pontos de Elo. Não houve treino adicional, nem parâmetro novo: apenas o direito de pensar por 0,4 segundo antes de responder. É a mesma alavanca hoje explorada em modelos de linguagem sob o nome de *test-time compute*.

4.3 Dados humanos

A comparação direta rodou o mesmo laço partindo de uma rede inicializada por aprendizado supervisionado sobre partidas humanas (conjunto KGS) e partindo de pesos aleatórios. A versão supervisionada aprende mais rápido *no início* e prevê jogadas humanas com mais acurácia *o tempo todo*; a versão *tabula rasa* a ultrapassa em poucas horas e termina mais forte.

Atenção

Cuidado com a generalização apressada. Aqui existem duas condições raras: um *simulador perfeito* e uma *recompensa verificável* ao final. Onde faltam essas duas coisas — diálogo, medicina, direção autônoma — a conclusão “dados humanos são dispensáveis” não se transfere. O que se transfere é a observação mais fina: imitação carrega o teto do imitado, e o auto-jogo não tem esse teto *quando existe um juiz confiável*.

4.4 Um algoritmo, três jogos

O AlphaZero removeu o que restava de específico do Go — simetrias do tabuleiro, torneio de avaliação entre gerações — e rodou o mesmo código em xadrez, shogi e Go.

	AlphaZero	Stockfish 8
Busca	MCTS guiada por rede	alfa-beta com poda
Posições por segundo	$\approx 6 \times 10^4$	$\approx 6 \times 10^7$
Conhecimento embutido	regras do xadrez	aberturas, avaliação, finais

No confronto de mil partidas publicado na *Science* em 2018: 155 vitórias, 6 derrotas, 839 empates. O treino levou 700 000 passos de gradiente, e o AlphaZero ultrapassou o Stockfish em cerca de quatro horas.

Mil vezes menos posições examinadas, e ainda assim vitória. A rede não acelera a busca; ela escolhe muito melhor o que examinar. Buscar menos e melhor bateu buscar muito e cego.

Atenção

As condições da partida foram legitimamente contestadas: versão do Stockfish, hardware assimétrico, controle de tempo, ausência de livro de aberturas. Em partidas com desvantagem de tempo, o AlphaZero se manteve superior até a proporção de 10:1. A comparação é informativa sobre *paradigmas de busca*; não é uma medição limpa de qual programa é mais forte.

5 Além do tabuleiro

A receita, escrita sem menção a jogos: *se um problema pode ser formulado como sequência de escolhas com um simulador que verifica o resultado, então uma rede que propõe candidatos somada a uma busca que testa candidatos pode superar a heurística humana, mesmo em espaços astronômicos*. A única peça que muda entre domínios é o simulador.

5.1 AlphaTensor

Multiplicar duas matrizes $n \times n$ equivale a decompor um tensor $\mathcal{T}_n$ em soma de tríades de posto 1; o número de parcelas é o número de multiplicações escalares. Cada ação do jogo é uma tríade, e o episódio termina quando o tensor residual zera.

Problema	melhor conhecido	AlphaTensor	observação
2×2 (Strassen, 1969)	7	7	redescoberto do zero
3×3 (Laderman, 1976)	23	23	redescoberto do zero
4×4 sobre $\mathbb{Z}_2$	49	47	primeiro avanço desde 1969
4×5 por 5×5 (reais)	80	76	
5×5 (reais)	98	96	

Duas ressalvas que a cobertura na imprensa perdeu. O resultado do 4×4 vale em aritmética modular ($\mathbb{Z}_2$), não sobre os reais. E, meses depois, Kauers e Moosbauer baixaram o caso 5×5 sobre GF(2) a 95 multiplicações usando busca combinatória clássica — a última palavra ainda não é da máquina.

Um detalhe metodológico interessante: quando a recompensa foi modificada para penalizar o *tempo medido* numa GPU V100 específica, o sistema encontrou algoritmos 10–20% mais rápidos *naquele hardware*. Otimalidade aqui é relativa à máquina, não uma propriedade matemática.

5.2 AlphaDev

No *AssemblyGame*, o estado é o programa em assembly construído até agora, a ação é acrescentar uma instrução, e a recompensa combina correção (testada contra entradas) com latência.

O alvo foram *sort3*, *sort4* e *sort5* — rotinas chamadas por dentro de qualquer ordenação maior. O sistema descobriu duas sequências novas, batizadas de *swap* e *copy* do AlphaDev, que economizam uma instrução cada vez que são aplicadas; para *VarSort4* encontrou um algoritmo com estrutura diferente e 29 instruções a menos que o de referência humano. Os resultados foram integrados à *libc++* do LLVM — a primeira mudança nessa parte da biblioteca em mais de uma década, e a primeira originada de aprendizagem por reforço.

Os ganhos: até 70% em sequências de cinco elementos, $\approx 1,7\%$ em sequências acima de 250 000 elementos, e $\approx 30\%$ em funções de *hash* na faixa de 9 a 16 bytes. A discrepância entre 70% e 1,7% não é contradição: as rotinas otimizadas são as *folhas* de algoritmos maiores. Melhorar a folha melhora muito a folha e pouco o todo — mas a folha é chamada trilhões de vezes por dia.

5.3 MuZero

AlphaZero exige um simulador perfeito. O MuZero remove essa exigência aprendendo um modelo — mas apenas o suficiente para planejar. Três funções são treinadas conjuntamente: representação h (observação $\mapsto$ estado latente), dinâmica $g((s^k, a) \mapsto (s^{k+1}, r^k))$, no espaço latente) e predição $f(s^k \mapsto (p, v))$. A MCTS roda inteiramente no espaço latente.

Intuição

Modelo suficiente para decidir, não para prever. Nada obriga s^k a reconstruir a observação: o modelo é treinado apenas para que política, valor e recompensa saiam certos. É a diferença entre um modelo *gerativo* do mundo e um modelo *orientado ao valor* — e prever pixels é caro e, em boa parte, irrelevante para a decisão.

6 Um olhar crítico: o UCT é o algoritmo certo?

6.1 A pergunta incômoda

O UCT trata cada nó como um bandit e usa uma cota superior de confiança para escolher o que simular. Por que isso é estranho?

Em um bandit *real*, cada braço puxado custa: a recompensa perdida é perda de verdade, e minimizar arrependimento cumulativo é exatamente o objetivo. Dentro da simulação, nada disso vale. Puxar mil vezes um braço ruim custa apenas tempo de CPU; a “recompensa” simulada não entra no bolso de ninguém. O objetivo correto não é acumular recompensa simulada — é *tomar a melhor decisão na raiz ao final do orçamento*.

Arrependimento cumulativo	Arrependimento simples
$R_n = \sum_{t=1}^n (\mu^* - \mu_{A_t})$	$r_n = \mu^* - \mu_{\hat{A}_n}$
o que se pagou pelo caminho	a qualidade da recomendação final
objetivo quando as ações são reais	objetivo quando as ações são simuladas

Bubeck, Munos e Stoltz (2009): existe um conflito

Quanto mais um algoritmo minimiza o arrependimento *cumulativo*, pior fica seu arrependimento *simples* no pior caso. Alocação uniforme atinge $r_n = O(e^{-cn})$. Já para qualquer estratégia com arrependimento cumulativo limitado por Cn^a uniformemente sobre as instâncias, existe uma instância em que $r_n \geq \frac{1}{2K} \exp(-C'n^a)$. Para o UCB, com $R_n = O(\log n)$, o expoente efetivo é logarítmico: o arrependimento simples decai apenas *polinomialmente* em n .

A intuição do teorema é direta e vale mais que o enunciado. Para ter R_n pequeno, é preciso *parar* de amostrar os braços que parecem ruins — e é exatamente amostrá-los que traria certeza sobre qual é o melhor. Os dois objetivos puxam em direções opostas.

6.2 O problema metanível

A pergunta correta não é “qual ação simular para ganhar mais recompensa simulada”, e sim:

Formulação metanível

Qual é a próxima *computação* a executar, dado que ela custa tempo e que seu único benefício é a chance de *mudar* a decisão da raiz?

Hay, Russell, Tolpin e Shimony (2012) mostram que isto é, literalmente, um MDP: os estados são estados de crença sobre os valores das ações, as ações são computações, e a recompensa é o valor da decisão final menos o custo do tempo gasto. A consequência prática mais útil é um critério de valor da informação:

Uma computação só vale a pena se puder mudar a jogada escolhida.

Refinar a estimativa do braço que já está em último lugar é desperdício, por mais incerta que essa estimativa seja — a incerteza não é o critério, a possibilidade de reversão do ranking é. Algoritmos desenhados para esse objetivo — BRUE, SHOT, e variantes de *sequential halving* em árvores — superam o UCT em vários cenários de orçamento fixo.

Intuição

Curiosamente, o PUCT do AlphaZero se afasta do UCT *na direção certa*, ainda que por acidente histórico. O decaimento $N(s, a)^{-1}$ é mais agressivo que o $N(s, a)^{-1/2}$ do UCT, e o

(continua)

crescimento $\sqrt{N(s)}$ é muito mais forte que $\sqrt{\log N(s)}$. O efeito líquido é distribuir visitas de forma mais parecida com uma *política* do que com uma cota de confiança — que é precisamente o que a leitura de Grill et al. formaliza.

6.3 A parede da profundidade: um experimento

O argumento teórico fica mais convincente com um exemplo executável. Considere a **árvore-armadilha**, no espírito das construções de Coquelin e Munos (2007):

Algoritmo — Árvore-armadilha de profundidade D

Em cada profundidade $d \in \{0, \dots, D-1\}$ há duas ações:

- ▶ *parar*: termina imediatamente com valor $0,9 - 0,4d/D$;
- ▶ *seguir*: vai para a profundidade $d+1$.

Chegar a $d = D$ vale 1,0. A política ótima é seguir sempre, com valor 1,0; a tentação está no começo, e a recompensa está no fim. Rollouts a partir de um nó folha escolhem *parar* ou *seguir* com probabilidade 1/2, de modo que alcançam a folha profunda com probabilidade $2^{-(D-d)}$.

Medindo a probabilidade de a raiz recomendar a ação ótima, com MCTS/UCT clássica e escolhendo o melhor c para cada célula:

D	$K = 200$	$K = 1\,000$	$K = 5\,000$	$K = 20\,000$
4	0,85	1,00	1,00	1,00
6	0,00	0,20	1,00	1,00
8	0,00	0,00	0,00	0,00
10	0,00	0,00	0,00	0,00

Com $D = 8$ nem 50 000 simulações resolvem. Duas causas se somam.

(i) A rollout aleatória quase nunca vê o valor bom. A probabilidade de alcançar a folha profunda decai como 2^{-D} . O que a busca observa, na esmagadora maioria das simulações, são os valores de *parar*.

(ii) O UCT retropropaga a média, não o máximo. Como observado na Seção 1.5, a média é tomada sobre uma política não estacionária. Ela fica contaminada por exploração antiga, e o atraso de cada nível se compõe ao subir. Coquelin e Munos exibem famílias de árvores em que o UCT precisa de um número de amostras *exponencial em uma exponencial* na profundidade. A garantia de consistência assintótica é verdadeira e inútil.

6.4 O ajuste de c não é secundário

Mesma árvore com $D = 6$ e orçamento fixo de 2 000 simulações, variando apenas a constante de exploração:

c	0,1	0,2	0,3	0,5	0,7	1,0	1,4
acerto	0,000	0,000	0,125	0,270	0,650	0,000	0,000

A curva não é monotônica nem suave. Pouca exploração nunca sai da armadilha rasa; muita exploração contamina de tal modo as médias retropropagadas que a raiz nunca enxerga a van-

tagem do caminho profundo. Note também que c precisa ser calibrado à *escala dos valores*: aqui os retornos vivem em $[0,5; 1,0]$, e um $c = 1,4$ que seria razoável para recompensas em $[0, 10]$ é aqui destrutivo.

Isto explica, em retrospecto, os 94 e 103 pontos de Elo obtidos no AlphaGo apenas com otimização bayesiana dos hiperparâmetros da busca.

6.5 Então por que a MCTS funciona tão bem na prática?

Quatro razões, em ordem de importância.

- 1. A rollout foi eliminada.** A causa (i) da parede da profundidade desaparece quando a folha é avaliada por v_θ em vez de por uma partida aleatória. O bom valor não precisa mais ser *encontrado* por sorte: ele é *previsto*.
- 2. O prior poda a largura.** Das ≈ 250 jogadas legais no Go, o PUCT examina seriamente uma dúzia. O fator de ramificação efetivo despenca.
- 3. As árvores reais não são adversárias.** Nas armadilhas construídas por Coquelin e Munos o valor está escondido atrás de um caminho estreito e único; em Go e xadrez, posições boas costumam ter vizinhas boas. Existe gradiente a seguir.
- 4. O laço de treino conserta a heurística.** Onde v_θ erra sistematicamente, o auto-jogo gera dados exatamente ali, porque a busca continua levando o agente àquelas posições.

A leitura para levar

A MCTS moderna funciona *apesar* de o UCT ser questionável, e não por causa dele. O peso saiu da regra de seleção e foi para a qualidade do avaliador de folha. É por isso que “mais simulações” rende cada vez menos, enquanto “rede melhor” continua rendendo. Consequência prática: ao diagnosticar um planejador que decide mal, suspeite primeiro do avaliador de folha e do prior, e só depois da constante de exploração ou do orçamento.

Para discutir em aula

O critério metanível diz que uma computação só vale a pena se puder mudar a decisão. Aplicado ao AlphaZero, ele sugeriria interromper a busca assim que a jogada mais visitada se tornasse inalcançável pelas demais dentro do orçamento restante — uma parada antecipada barata e correta. Por que isso raramente é feito? E qual seria o análogo dessa ideia em um modelo de linguagem que “pensa” por mais tempo antes de responder?

7 Formulário

Objeto	Expressão
Busca MC simples	$\hat{Q}(s_t, a) = \frac{1}{K} \sum_k G_t^k; \quad a_t = \arg \max_a \hat{Q}(s_t, a)$
Tamanho do expectimax	$(\mathcal{S} \mathcal{A})^H$
Seleção UCT	$\arg \max_a [\bar{G}(s, a) + c\sqrt{\log N(s)/N(s, a)}]$
Seleção PUCT	$\arg \max_a [Q(s, a) + c_{\text{puct}}P(s, a)\sqrt{N(s)/(1 + N(s, a))}]$
Retropropagação	$N += 1; W += \pm v; Q = W/N$
Ruído na raiz	$P(s_0, a) = (1 - \varepsilon)p_\theta(s_0)_a + \varepsilon\eta_a, \quad \eta \sim \text{Dir}(\alpha), \quad \varepsilon = 0,25$
Política da raiz	$\pi(a s_0) = N(s_0, a)^{1/\tau} / \sum_b N(s_0, b)^{1/\tau}$
Perda do AlphaZero	$\ell(\theta) = (z - v_\theta(s))^2 - \pi^\top \log p_\theta(s) + c\ \theta\ ^2$
Forma fechada de Grill	$\bar{\pi}_a = \lambda_N p_{\theta,a}/(\alpha - q_a), \quad \lambda_N = c_{\text{puct}}\sqrt{N}/(N + \mathcal{A})$
Arrependimento simples	$r_n = \mu^* - \mu_{\hat{A}_n} \quad (\text{contra } R_n = \sum_t (\mu^* - \mu_{A_t}))$
Escala Elo	$p(\text{vitória}) = (1 + 10^{-\Delta/400})^{-1}; \quad \Delta = 200 \Rightarrow p \approx 0,76$

Sistema	Avaliação da folha	Modelo
MCTS clássica / UCT	rollout aleatória	dado (simulador)
AlphaGo (2016)	$\lambda \text{ rollout} + (1 - \lambda) v_\theta$	dado
AlphaGo Zero (2017)	v_θ apenas	dado
AlphaZero (2018)	v_θ apenas	dado, três jogos
MuZero (2020)	v_θ apenas	aprendido, latente

8 Exercícios

Exercício L14E1 — PUCT à mão (com resposta)

Raiz com três ações, priors $P = (0,5; 0,3; 0,2)$, valores das folhas $v_\theta = (0,20; 0,60; 0,10)$ e $c_{\text{puct}} = 1,0$. Use a convenção $\sqrt{N(s)} \rightarrow 1$ na primeira simulação. Execute oito simulações à mão, preenchendo Q , U e $Q + U$ a cada passo, e reporte N final e π com $\tau = 1$.

Demonstração

Resposta. A sequência de ações escolhidas é $a_1, a_1, a_1, a_2, a_2, a_2, a_2, a_2$.

k	Q			U			escolhe
	a_1	a_2	a_3	a_1	a_2	a_3	
1	0,000	0,000	0,000	0,500	0,300	0,200	a_1
2	0,200	0,000	0,000	0,250	0,300	0,200	a_1
3	0,200	0,000	0,000	0,236	0,424	0,283	a_1
4	0,200	0,000	0,000	0,217	0,520	0,346	a_2
5	0,200	0,600	0,000	0,250	0,300	0,400	a_2
6	0,200	0,600	0,000	0,280	0,224	0,447	a_2
7	0,200	0,600	0,000	0,306	0,184	0,490	a_2
8	0,200	0,600	0,000	0,331	0,159	0,529	a_2

Final: $N = (3, 5, 0)$ e $\pi = (0,375; 0,625; 0)$.

(continua)

Observe que a_1 é escolhida três vezes seguidas apesar de já ter devolvido 0,20 na primeira: com c_{puct} menor e prior mais concentrado que no exemplo da Seção 2.3, a busca demora mais a migrar. E a_3 nunca é visitada, embora seu U seja o maior dos três a partir de $k = 6$ — o que falta é orçamento, não incentivo.

Exercício L14E2 — Os dois regimes da forma fechada

Usando $\bar{\pi}_a = \lambda p_a / (\alpha - q_a)$ com α resolvendo $\sum_a \lambda p_a / (\alpha - q_a) = 1$:

1. Mostre que $\lambda \rightarrow \infty$ implica $\bar{\pi} \rightarrow p$.
2. Mostre que $\lambda \rightarrow 0^+$ implica $\bar{\pi} \rightarrow$ indicadora do $\arg \max$ de q (suponha o máximo único).
3. Com $p = (0,6; 0,3; 0,1)$ e $q = (0,10; 0,55; -0,40)$, calcule numericamente α e $\bar{\pi}$ para $\lambda = 0,0374$ e confira contra a tabela da Seção 2.8.

Exercício L14E3 — Temperatura e entropia

Com $N = (5, 11, 0)$:

1. Calcule π e $H[\pi]$ para $\tau \in \{1; 0,5; 0,25; 0,1\}$.
2. Mostre que $\lim_{\tau \rightarrow 0^+} \pi$ é a indicadora do $\arg \max_a N$ e que $\lim_{\tau \rightarrow \infty} \pi$ é uniforme sobre as ações visitadas.
3. O AlphaZero usa $\tau = 1$ nas 30 primeiras jogadas. Argumente por que usar $\tau = 1$ na partida inteira degradaria simultaneamente a força de jogo e a qualidade do alvo de valor.

Exercício L14E4 — Densidade de informação no auto-jogo

Modele o resultado de uma partida como Bernoulli(p), com p a probabilidade de vitória do agente que está treinando.

1. Calcule $H(p)$ em bits para $p \in \{0,5; 0,75; 0,9; 0,99\}$ e o número de partidas necessárias para acumular 1 bit.
2. Suponha um adversário fixo contra o qual o agente vence 99% das vezes. Quantas vezes mais partidas são necessárias, em relação ao auto-jogo, para obter a mesma informação sobre o parâmetro?
3. O argumento acima trata só do alvo de *valor*. Explique por que o alvo de *política* não sofre a mesma degradação — e por que, ainda assim, um adversário desbalanceado prejudica o treino.

Exercício L14E5 — Elo e o valor da busca

Use $p = (1 + 10^{-\Delta/400})^{-1}$.

1. Confirme que $\Delta = 200$ dá $p \approx 0,76$.
2. A rede crua do AlphaGo Zero tem Elo 3 055 e o sistema completo, 5 185. Calcule p e o número esperado de partidas até a rede crua vencer uma.
3. A escala Elo é logarítmica na razão de chances. Discuta em que sentido dizer que “a busca vale 2 130 Elo” é mais informativo — e em que sentido é menos — do que dizer que ela vence 99,9995% das partidas.

Exercício L14E6 — A árvore-armadilha (implementação)

Implemente a árvore-armadilha do algoritmo da Seção 6.3 e a MCTS/UCT com rollout aleatória.

1. Reproduza a tabela de acerto na raiz para $D \in \{4, 6, 8\}$ e $K \in \{200; 1\,000; 5\,000\}$, varrendo

(continua)

$c \in \{0,3; 0,5; 0,7; 1,0\}$.

2. Substitua a rollout por um avaliador $\hat{v}(d) = V^*(d) + \epsilon$ com $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Trace a curva de acerto *versus* σ para $D = 8$ e $K = 1\,000$. A partir de que σ o avaliador deixa de ajudar?
3. Instrumente o código para registrar, a cada 100 simulações, o valor $Q(\text{raiz}, \text{seguir})$. Verifique empiricamente que ele converge para 1,0 *por baixo*, e relacione isso com a discussão sobre média *versus* máximo da Seção 1.5.

Exercício L14E7 — Min-max implícito

No AlphaZero, $v_\theta(s)$ é sempre o valor do ponto de vista do jogador da vez, e a retropropagação alterna o sinal a cada nível.

1. Escreva a recorrência de $Q(s, a)$ resultante e mostre que, no limite de muitas simulações, a árvore implementa uma avaliação min-max.
2. Onde exatamente essa construção usa a hipótese de *soma zero*? Descreva o que quebra em um jogo de dois jogadores com recompensas arbitrárias.
3. Como você adaptaria o esquema para três jogadores? (Dica: pense no que substitui o par (max, min).)

Exercício L14E8 — Quando não usar MCTS

Para cada cenário, diga se a MCTS é uma boa escolha e justifique em uma frase.

1. Controle de um pêndulo invertido, estado contínuo em $\mathbb{R}^4$, ação contínua, horizonte de 500 passos, simulador disponível.
2. Escalonamento de tarefas com 8 máquinas e 20 tarefas, modelo exato conhecido, horizonte 20.
3. Recomendação de conteúdo com milhões de itens, sem simulador confiável do comportamento do usuário.
4. Planejamento de rotas em um grafo de 10^7 nós com custos conhecidos e determinísticos.

Exercício L14E9 — Custo de decisão *versus* custo de treino

O AlphaGo Zero usa 1 600 simulações por jogada, a $\approx 0,4$ s por busca, e gerou 4,9 milhões de partidas em três dias. Supondo 200 jogadas por partida:

1. Estime o número total de avaliações da rede na fase de auto-jogo.
2. Compare com o número de passos de gradiente ($\approx 700\,000$ no AlphaZero, com lotes de 4 096). Qual fase domina o custo?
3. Que consequência isso tem para quem quer reproduzir o experimento com orçamento acadêmico? Que parâmetro você reduziria primeiro, e o que se perde?

9 Autoavaliação

Marque o que você consegue fazer sem consultar o material.

1. Explicar por que jogar Go *não* é um problema de modelo desconhecido, e o que de fato torna o Go difícil.
2. Escrever a regra PUCT de memória e dizer o papel de cada um dos quatro fatores do termo U .
3. Listar três diferenças entre UCT e PUCT, e dizer qual delas explica por que o AlphaZero precisa de ruído de Dirichlet.
4. Explicar por que trocar a rollout pela rede de valor é a mesma troca de Monte Carlo por TD, incluindo o efeito sobre viés e variância.
5. Justificar por que a política da raiz usa contagens de visita e não $\arg \max_a Q$.
6. Derivar a forma fechada $y_a = \lambda p_a / (\alpha - q_a)$ e identificar os dois regimes-limite.
7. Explicar em que sentido a MCTS é um operador de melhoria de política, e por que isso transforma o laço de treino em iteração de política.
8. Argumentar, com o cálculo de entropia, por que o auto-jogo mantém a recompensa densa em informação.
9. Distinguir arrependimento simples de cumulativo e explicar por que o UCT otimiza o segundo dentro de um problema que pede o primeiro.
10. Descrever a parede da profundidade, suas duas causas, e por que a rede de valor elimina uma delas.

10 Leituras

Fontes primárias

- ▶ Silver, D. et al. *Mastering the game of Go with deep neural networks and tree search*. Nature 529, 484–489, 2016.
- ▶ Silver, D. et al. *Mastering the game of Go without human knowledge*. Nature 550, 354–359, 2017.
- ▶ Silver, D. et al. *A general reinforcement learning algorithm that masters chess, shogi and Go through self-play*. Science 362, 1140–1144, 2018.
- ▶ Schrittwieser, J. et al. *Mastering Atari, Go, chess and shogi by planning with a learned model*. Nature 588, 604–609, 2020.
- ▶ Fawzi, A. et al. *Discovering faster matrix multiplication algorithms with reinforcement learning*. Nature 610, 47–53, 2022.
- ▶ Mankowitz, D. et al. *Faster sorting algorithms discovered using deep reinforcement learning*. Nature 618, 257–263, 2023.

Fundamentos, análise e crítica

- ▶ Kocsis, L. e Szepesvári, C. *Bandit based Monte-Carlo planning*. ECML, 2006. (UCT)
- ▶ Coquelin, P.-A. e Munos, R. *Bandit algorithms for tree search*. UAI, 2007. (patologias do UCT)
- ▶ Bubeck, S., Munos, R. e Stoltz, G. *Pure exploration in multi-armed bandits problems*. ALT, 2009.
- ▶ Hay, N., Russell, S., Tolpin, D. e Shimony, S. *Selecting computations: theory and applications*. UAI, 2012.
- ▶ Browne, C. et al. *A survey of Monte Carlo tree search methods*. IEEE TCIAIG 4(1), 1–43, 2012.

- ▶ Anthony, T., Tian, Z. e Barber, D. *Thinking fast and slow with deep learning and tree search*. NeurIPS, 2017.
- ▶ Chen, Y. et al. *Bayesian optimization in AlphaGo*. arXiv:1812.06855, 2018.
- ▶ Grill, J.-B. et al. *Monte-Carlo tree search as regularized policy optimization*. ICML, 2020.